

Tredit - Decentralized E-commerce Platform

Tredit is a comprehensive decentralized e-commerce platform that combines traditional e-commerce features with blockchain technology to create a secure, transparent, and efficient marketplace.

Table of Contents

- [Overview](#)
- [Architecture](#)
- [Smart Contracts](#)
- [Frontend](#)
- [Backend](#)
- [Database](#)
- [Blockchain Integration](#)
- [Security](#)
- [Setup and Installation](#)
- [Development](#)
- [Deployment](#)
- [Testing](#)
- [Contributing](#)
- [License](#)
- [Documentation](#)
- [Troubleshooting](#)
- [Performance Optimization](#)
- [Monitoring and Logging](#)
- [API Documentation](#)
- [Smart Contract Documentation](#)
- [Frontend Documentation](#)
- [Backend Documentation](#)
- [Database Documentation](#)
- [Deployment Documentation](#)
- [Security Documentation](#)
- [Contributing Guidelines](#)
- [Code of Conduct](#)

- [Changelog](#)
- [Roadmap](#)
- [FAQ](#)
- [Support](#)
- [Acknowledgments](#)

Overview

Tredit is a decentralized e-commerce platform that leverages blockchain technology to provide:

Core Features

- Secure payment processing
- Escrow services for goods
- Dispute resolution system
- Business verification
- User profile management
- Product management
- Order tracking
- Review and rating system
- Chat and messaging

Key Benefits

- Decentralized architecture
- Transparent transactions
- Secure escrow system
- Automated dispute resolution
- Trustless business verification
- Immutable product records
- Real-time order tracking
- Community-driven reviews
- Secure messaging

Target Users

- Individual sellers
- Business owners

- Buyers
- Arbitrators
- Developers
- System administrators

Architecture

The platform is built using a modern tech stack:

Technology Stack

- Frontend: Next.js with TypeScript
- Backend: Node.js with Express
- Database: PostgreSQL (Supabase)
- Blockchain: Polygon Network
- Storage: IPFS (Pinata)
- Authentication: NextAuth.js
- Payment: Paystack

System Components

1. Smart Contracts
 - Dispute resolution
 - Business management
 - User profiles
 - Payment processing
 - Escrow management
2. Frontend Application
 - User interface
 - Business dashboard
 - Product management
 - Order tracking
 - Dispute resolution
 - Chat system
3. Backend API
 - RESTful endpoints
 - WebSocket support
 - Authentication

- Business logic
- Data validation

4. Database

- User data
- Business information
- Product catalog
- Order records
- Transaction history
- Dispute records
- Chat messages

5. Blockchain Integration

- Smart contract interaction
- Transaction processing
- Payment handling
- Escrow management
- Dispute resolution

6. Storage System

- IPFS integration
- File management
- Content addressing
- Data persistence

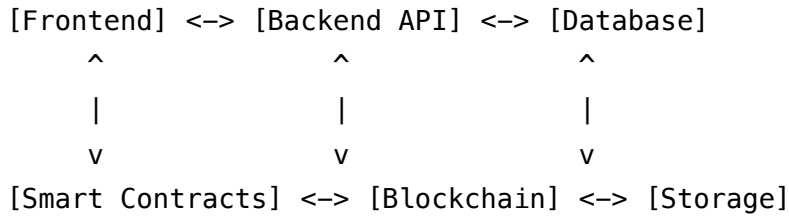
7. Authentication System

- JWT tokens
- OAuth providers
- Session management
- Role-based access

8. Payment Gateway

- Crypto payments
- Fiat integration
- Escrow management
- Refund processing

System Architecture Diagram



Smart Contracts

The platform uses several smart contracts to handle different aspects of the system:

1. Dispute Contract (`Dispute.sol`)

- Handles dispute creation and resolution
- Manages evidence submission
- Tracks dispute status
- Supports multiple resolution types

Key Functions

- `createDispute`
- `addEvidence`
- `resolveDispute`
- `closeDispute`
- `getDispute`
- `getEvidence`

Events

- `DisputeCreated`
- `EvidenceAdded`
- `DisputeResolved`
- `DisputeClosed`

2. Business Contract (`Business.sol`)

- Manages business profiles
- Handles business verification

- Tracks business status
- Stores business metadata on IPFS

Key Functions

- `createBusiness`
- `updateBusiness`
- `deleteBusiness`
- `setBusinessStatus`
- `getBusiness`
- `getBusinessesByOwner`

Events

- `BusinessCreated`
- `BusinessUpdated`
- `BusinessDeleted`
- `BusinessStatusChanged`

3. User Profile Contract (`UserProfile.sol`)

- Manages user profiles
- Links users to businesses
- Stores profile data on IPFS
- Handles profile status

Key Functions

- `createOrUpdateProfile`
- `setProfileStatus`
- `addBusiness`
- `removeBusiness`
- `getProfile`

Events

- `ProfileCreated`
- `ProfileUpdated`
- `ProfileStatusChanged`

4. Payment Contract (`Payment.sol`)

- Processes payments
- Supports multiple tokens
- Handles fiat and crypto payments
- Manages payment status

Key Functions

- `initiateBlockchainPayment`
- `completePayment`
- `recordFiatPayment`
- `getPayment`
- `addSupportedToken`
- `removeSupportedToken`

Events

- `PaymentInitiated`
- `PaymentCompleted`
- `TokenAdded`
- `TokenRemoved`

5. Goods Escrow Contract (`GoodsEscrow.sol`)

- Manages escrow for goods
- Handles delivery confirmation
- Processes refunds
- Manages dispute resolution

Key Functions

- `initiatePayment`
- `confirmDelivery`
- `raiseDispute`
- `resolveDispute`
- `requestRefund`

Events

- `PaymentDeposited`

- DeliveryConfirmed
- DisputeRaised
- DisputeResolved
- RefundIssued

Frontend

The frontend is built with Next.js and TypeScript, organized into feature-based directories:

Key Components

- Authentication
 - Login/Register
 - OAuth integration
 - Session management
 - Role-based access
- Dashboard
 - User overview
 - Business metrics
 - Order tracking
 - Dispute management
- Payment Processing
 - Crypto payments
 - Fiat integration
 - Transaction history
 - Refund management
- Business Management
 - Profile setup
 - Product management
 - Order processing
 - Analytics
- Product Management
 - Listing creation
 - Inventory tracking
 - Price management
 - Category organization
- Order Management
 - Order creation

- Status tracking
 - Delivery management
 - History view
- Dispute Resolution
 - Dispute creation
 - Evidence submission
 - Resolution tracking
 - Communication
- Chat System
 - Real-time messaging
 - File sharing
 - Notification system
 - History tracking

Directory Structure

```
src/
├─ app/
│   │   └─ auth/
│   │       │   └─ login/
│   │       │   └─ register/
│   │       └─ oauth/
│   └─ dashboard/
│       │   └─ overview/
│       │   └─ metrics/
│       └─ settings/
├─ payment/
│   │   └─ crypto/
│   │   └─ fiat/
│   └─ history/
├─ business/
│   │   └─ profile/
│   │   └─ products/
│   └─ orders/
├─ product/
│   │   └─ create/
│   │   └─ edit/
│   └─ list/
├─ order/
│   │   └─ create/
│   │   └─ track/
│   └─ history/
├─ dispute/
│   │   └─ create/
│   │   └─ evidence/
│   └─ resolution/
├─ chat/
│   │   └─ messages/
│   │   └─ files/
│   └─ notifications/
└─ api/
    │   └─ auth/
    │   └─ business/
    │   └─ payment/
    │   └─ product/
    │   └─ order/
    └─ dispute/
```

```
| | | └─ chat/
| | └─ layout.tsx
| | └─ providers.tsx
└─ components/
    | └─ common/
    | └─ forms/
    | └─ modals/
    | └─ widgets/
└─ hooks/
    | └─ auth/
    | └─ business/
    | └─ payment/
    | └─ chat/
└─ lib/
    | └─ api/
    | └─ blockchain/
    | └─ storage/
    | └─ utils/
└─ types/
    | └─ api/
    | └─ blockchain/
    | └─ models/
```

Key Technologies

- Next.js 13+
- TypeScript 5+
- React 18+
- Tailwind CSS
- Web3.js
- Ethers.js
- [Socket.io](#)
- React Query
- Zustand
- React Hook Form
- Zod

Backend

The backend is built with Node.js and Express, providing RESTful APIs for:

API Endpoints

- User Management
 - Authentication
 - Profile management
 - Role management
 - Session handling
- Business Operations
 - Profile management
 - Product management
 - Order processing
 - Analytics
- Product Management
 - CRUD operations
 - Category management
 - Inventory tracking
 - Price management
- Order Processing
 - Order creation
 - Status updates
 - Delivery tracking
 - History management
- Payment Handling
 - Crypto payments
 - Fiat integration
 - Transaction processing
 - Refund management
- Dispute Resolution
 - Dispute creation
 - Evidence management
 - Resolution processing
 - Communication
- Chat Functionality
 - Message handling
 - File sharing
 - Notification system
 - History management

API Routes

```
/api/  
├─ auth/  
│   ├── login  
│   ├── register  
│   ├── oauth  
│   └─ session  
├─ business/  
│   ├── profile  
│   ├── products  
│   ├── orders  
│   └─ analytics  
├─ payment/  
│   ├── crypto  
│   ├── fiat  
│   └─ history  
├─ product/  
│   ├── create  
│   ├── update  
│   ├── delete  
│   └─ list  
├─ order/  
│   ├── create  
│   ├── update  
│   ├── track  
│   └─ history  
├─ dispute/  
│   ├── create  
│   ├── evidence  
│   ├── resolve  
│   └─ history  
└─ chat/  
    ├── messages  
    ├── files  
    └─ notifications
```

Key Technologies

- Node.js 18+
- Express 4+
- TypeScript 5+

- Prisma
- WebSocket
- JWT
- Bcrypt
- Multer
- [Socket.io](#)
- Redis
- Bull

Database

The database schema is defined using Prisma and includes models for:

Key Models

- User
 - Profile information
 - Authentication data
 - Role management
 - Session tracking
- Business
 - Profile data
 - Verification status
 - Product catalog
 - Order history
- Product
 - Basic information
 - Pricing data
 - Inventory tracking
 - Category management
- Order
 - Transaction details
 - Status tracking
 - Delivery information
 - Payment history
- Transaction
 - Payment details
 - Status tracking

- Refund information
 - History management
- Dispute
 - Case information
 - Evidence tracking
 - Resolution status
 - Communication history
- Review
 - Rating data
 - Comment management
 - History tracking
 - Moderation status
- Chat
 - Message history
 - File attachments
 - User information
 - Status tracking

Database Schema

```
model User {  
  id          String    @id @default(uuid())  
  email       String    @unique  
  password    String  
  role        Role      @default(USER)  
  profile     Profile?  
  businesses  Business[]  
  orders      Order[]  
  disputes    Dispute[]  
  reviews     Review[]  
  chats       Chat[]  
  createdAt   DateTime  @default(now())  
  updatedAt   DateTime  @updatedAt  
}
```

```
model Business {  
  id          String    @id @default(uuid())  
  name        String  
  owner       User      @relation(fields: [ownerId], references: [id])  
  ownerId     String  
  products    Product[]  
  orders      Order[]  
  disputes    Dispute[]  
  reviews     Review[]  
  createdAt   DateTime  @default(now())  
  updatedAt   DateTime  @updatedAt  
}
```

```
model Product {  
  id          String    @id @default(uuid())  
  name        String  
  description  String  
  price       Decimal  
  business    Business  @relation(fields: [businessId], references: [id])  
  businessId  String  
  orders      Order[]  
  reviews     Review[]  
  createdAt   DateTime  @default(now())  
  updatedAt   DateTime  @updatedAt  
}
```



```

model Order {
  id          String    @id @default(uuid())
  user        User      @relation(fields: [userId], references: [id])
  userId      String
  business    Business  @relation(fields: [businessId], references: [id])
  businessId  String
  product     Product   @relation(fields: [productId], references: [id])
  productId   String
  status      OrderStatus
  transactions Transaction[]
  disputes    Dispute[]
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
}

```

```

model Transaction {
  id          String    @id @default(uuid())
  order       Order     @relation(fields: [orderId], references: [id])
  orderId     String
  amount      Decimal
  status      TransactionStatus
  type        TransactionType
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
}

```

```

model Dispute {
  id          String    @id @default(uuid())
  order       Order     @relation(fields: [orderId], references: [id])
  orderId     String
  user        User      @relation(fields: [userId], references: [id])
  userId      String
  business    Business  @relation(fields: [businessId], references: [id])
  businessId  String
  status      DisputeStatus
  evidence    Evidence[]
  resolution  Resolution?
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
}

```

```

model Review {
  id          String    @id @default(uuid())

```

```

    user      User      @relation(fields: [userId], references: [id])
    userId    String
    business  Business  @relation(fields: [businessId], references: [id])
    businessId String
    product   Product   @relation(fields: [productId], references: [id])
    productId String
    rating    Int
    comment    String?
    createdAt DateTime @default(now())
    updatedAt DateTime @updatedAt
}

```

```

model Chat {
    id          String    @id @default(uuid())
    user        User      @relation(fields: [userId], references: [id])
    userId      String
    messages    Message[]
    createdAt   DateTime @default(now())
    updatedAt   DateTime @updatedAt
}

```

```

model Message {
    id          String    @id @default(uuid())
    chat        Chat      @relation(fields: [chatId], references: [id])
    chatId      String
    content      String
    attachments  Attachment[]
    createdAt   DateTime @default(now())
    updatedAt   DateTime @updatedAt
}

```

```

model Attachment {
    id          String    @id @default(uuid())
    message     Message   @relation(fields: [messageId], references: [id])
    messageId   String
    type        AttachmentType
    url         String
    createdAt   DateTime @default(now())
    updatedAt   DateTime @updatedAt
}

```

Blockchain Integration

The platform integrates with the Polygon network for:

Smart Contract Deployment

- Contract compilation
- Deployment scripts
- Network configuration
- Contract verification

Transaction Processing

- Payment handling
- Escrow management
- Dispute resolution
- Status tracking

Network Configuration

- Mainnet: Polygon
 - Chain ID: 137
 - RPC URL: <https://polygon-rpc.com>
 - Explorer: <https://polygonscan.com>
- Testnet: Polygon Amoy
 - Chain ID: 80002
 - RPC URL: <https://polygon-amoy.infura.io/v3/...>
 - Explorer: <https://www.oklink.com/amoy>
- Local: Hardhat
 - Chain ID: 31337
 - RPC URL: <http://127.0.0.1:8545>
 - Explorer: N/A

Contract Addresses

- Token Contract: 0x93dEaFdC8F05Fe98795a7Dc9Bad2Db5D9EBB7627
- User Profile Contract: 0xE26BD5E346febFD582Ca3089a253f11E9fbC3d51
- Business Contract: 0xA97Fe725ED7270273f1cD95f15FdB9A1a1aAD8fd
- Dispute Contract: 0xbd60102Ea9e8bCa388159fce4dFc90e2D54085Ae

- Payment Contract: 0x2Dc079857d7aac7e145E098C1d9B8686bFA4Ae3b
- Escrow Contract: 0xB945aC65408beBBE0eA082890913aCFfD480b461

Security

The platform implements multiple security measures:

Smart Contract Security

- Access control (Ownable)
- Reentrancy protection
- Input validation
- Event logging
- Gas optimization
- Error handling
- State management
- Upgrade patterns

Application Security

- JWT authentication
- Rate limiting
- CORS protection
- Session management
- Input sanitization
- XSS prevention
- CSRF protection
- SQL injection prevention

Data Security

- IPFS encryption
- Secure storage
- Data validation
- Access control
- Backup systems
- Disaster recovery
- Audit logging

- Compliance measures

Setup and Installation

Prerequisites

- Node.js (v18 or higher)
- PostgreSQL
- MetaMask or similar wallet
- Git
- Docker (optional)
- Redis (optional)

Environment Setup

1. Clone the repository

```
git clone https://github.com/your-username/tredit.git
cd tredit
```

2. Install dependencies

```
npm install
```

3. Set up environment variables

```
cp .env.example .env
# Edit .env with your configuration
```

4. Initialize the database

```
npx prisma migrate dev
```

5. Start the development server

```
npm run dev
```

Docker Setup

1. Build the Docker image

```
docker build -t tredit .
```

2. Run the container

```
docker run -p 3000:3000 tredit
```

Development

Smart Contract Development

1. Install Hardhat

```
npm install --save-dev hardhat
```

2. Compile contracts

```
npx hardhat compile
```

3. Run tests

```
npx hardhat test
```

4. Deploy contracts

```
npx hardhat run scripts/deploy.js --network polygonAmoy
```

Frontend Development

1. Start the development server

```
npm run dev
```

2. Build for production

```
npm run build
```

3. Run tests

```
npm run test
```

4. Lint code

```
npm run lint
```

Backend Development

1. Start the API server

```
npm run api:dev
```

2. Run tests

```
npm run test
```

3. Lint code

```
npm run lint
```

4. Generate API documentation

```
npm run docs:api
```

Deployment

Smart Contract Deployment

1. Configure network in `hardhat.config.js`

2. Deploy contracts

```
npx hardhat run scripts/deploy.js --network polygonAmoy
```

3. Verify contracts

```
npx hardhat verify --network polygonAmoy <CONTRACT_ADDRESS>
```

Frontend Deployment

1. Build the application

```
npm run build
```

2. Deploy to your hosting provider

```
npm run deploy
```

Backend Deployment

1. Configure environment variables

2. Deploy to your server

```
npm run deploy:api
```

Database Deployment

1. Run migrations

```
npx prisma migrate deploy
```

2. Seed the database

```
npx prisma db seed
```


Testing

Smart Contract Testing

- Unit tests
- Integration tests
- Security tests
- Gas optimization tests
- Upgrade tests

Frontend Testing

- Component tests
- Integration tests
- E2E tests
- Performance tests
- Accessibility tests

Backend Testing

- API tests
- Integration tests
- Performance tests
- Security tests
- Load tests

Contributing

1. Fork the repository
2. Create a feature branch
3. Commit your changes
4. Push to the branch
5. Create a Pull Request

Development Workflow

1. Create an issue
2. Assign the issue

3. Create a branch
4. Make changes
5. Write tests
6. Update documentation
7. Create PR
8. Review and merge

Code Style

- Follow TypeScript best practices
- Use ESLint and Prettier
- Write meaningful commit messages
- Document your code
- Write tests for new features

License

This project is licensed under the MIT License - see the [LICENSE](#) file for details.

Documentation

API Documentation

- REST API endpoints
- WebSocket events
- Authentication
- Error handling
- Rate limiting
- Pagination
- Filtering
- Sorting

Smart Contract Documentation

- Contract interfaces
- Function descriptions
- Event definitions
- State variables

- Security considerations
- Upgrade procedures
- Gas optimization
- Testing guidelines

Frontend Documentation

- Component library
- State management
- Routing
- Authentication
- API integration
- Styling guidelines
- Performance optimization
- Testing strategies

Backend Documentation

- API architecture
- Database schema
- Authentication
- Authorization
- Error handling
- Logging
- Monitoring
- Deployment

Database Documentation

- Schema design
- Relationships
- Indexes
- Migrations
- Seeding
- Backup
- Recovery
- Performance

Troubleshooting

Common Issues

- Installation problems
- Configuration errors
- Network issues
- Contract deployment
- Database connection
- API errors
- Frontend bugs
- Performance issues

Solutions

- Check logs
- Verify configuration
- Test network connection
- Validate contracts
- Check database
- Debug API
- Inspect frontend
- Monitor performance

Performance Optimization

Frontend

- Code splitting
- Lazy loading
- Image optimization
- Caching
- Bundle size
- Rendering performance
- Network requests
- State management

Backend

- Database optimization
- Caching
- Load balancing
- API optimization
- Error handling
- Logging
- Monitoring
- Scaling

Smart Contracts

- Gas optimization
- Storage efficiency
- Function optimization
- Event optimization
- Upgrade patterns
- Security measures
- Testing coverage
- Documentation

Monitoring and Logging

Frontend

- Error tracking
- Performance monitoring
- User analytics
- Session tracking
- Network monitoring
- Console logging
- Debug tools
- Testing coverage

Backend

- Error logging

- Performance metrics
- API monitoring
- Database monitoring
- Security monitoring
- User tracking
- System health
- Resource usage

Smart Contracts

- Event logging
- Transaction monitoring
- Gas usage
- Error tracking
- Security monitoring
- Upgrade tracking
- State changes
- Contract interactions

API Documentation

REST API

- Authentication
- User management
- Business operations
- Product management
- Order processing
- Payment handling
- Dispute resolution
- Chat functionality

WebSocket API

- Real-time updates
- Chat messages
- Order status
- Payment status

- Dispute updates
- Notifications
- System events
- User presence

Smart Contract Documentation

Contract Interfaces

- Function signatures
- Event definitions
- State variables
- Access control
- Security measures
- Upgrade patterns
- Gas optimization
- Testing guidelines

Deployment

- Network configuration
- Contract addresses
- Verification
- Testing
- Monitoring
- Upgrades
- Security
- Documentation

Frontend Documentation

Components

- Authentication
- Dashboard
- Payment
- Business

- Product
- Order
- Dispute
- Chat

State Management

- Global state
- Local state
- API integration
- WebSocket
- Caching
- Persistence
- Performance
- Testing

Backend Documentation

API Architecture

- REST endpoints
- WebSocket
- Authentication
- Authorization
- Error handling
- Rate limiting
- Caching
- Monitoring

Database

- Schema
- Migrations
- Seeding
- Backup
- Recovery
- Performance
- Security

- Monitoring

Deployment Documentation

Smart Contracts

- Network setup
- Contract deployment
- Verification
- Testing
- Monitoring
- Upgrades
- Security
- Documentation

Frontend

- Build process
- Deployment
- Hosting
- CDN
- SSL
- Monitoring
- Backup
- Recovery

Backend

- Server setup
- API deployment
- Database
- Caching
- Load balancing
- Monitoring
- Backup
- Recovery

Security Documentation

Smart Contracts

- Access control
- Reentrancy
- Input validation
- Event logging
- Gas optimization
- Error handling
- State management
- Upgrade patterns

Application

- Authentication
- Authorization
- Input validation
- XSS prevention
- CSRF protection
- SQL injection
- Rate limiting
- Monitoring

Data

- Encryption
- Storage
- Backup
- Recovery
- Access control
- Audit logging
- Compliance
- Monitoring

Contributing Guidelines

Development

- Code style
- Testing
- Documentation
- Review process
- Merge process
- Release process
- Versioning
- Changelog

Communication

- Issues
- Pull requests
- Discussions
- Documentation
- Code review
- Feedback
- Support
- Community

Code of Conduct

Behavior

- Professional
- Respectful
- Inclusive
- Collaborative
- Constructive
- Supportive
- Ethical
- Legal

Guidelines

- Communication
- Collaboration
- Feedback
- Support
- Documentation
- Testing
- Security
- Privacy

Changelog

Version History

- Major releases
- Minor updates
- Bug fixes
- Security patches
- Performance improvements
- New features
- Breaking changes
- Deprecations

Release Notes

- Changes
- Features
- Fixes
- Security
- Performance
- Documentation
- Migration
- Support

Roadmap

Future Features

- Mobile app
- Advanced analytics
- AI integration
- Multi-chain support
- Enhanced security
- Performance optimization
- User experience
- Community features

Development Plans

- Architecture
- Infrastructure
- Security
- Performance
- Scalability
- Reliability
- Maintainability
- Documentation

FAQ

General

- Platform overview
- Features
- Benefits
- Requirements
- Setup
- Usage
- Support
- Community

Technical

- Architecture
- Development
- Deployment
- Security
- Performance
- Testing
- Documentation
- Support

Business

- Use cases
- Benefits
- Requirements
- Setup
- Usage
- Support
- Community
- Resources

Support

Channels

- GitHub issues
- Documentation
- Community forum
- Email support
- Chat support
- Social media
- Blog
- Newsletter

Resources

- Documentation

- [Tutorials](#)
- [Guides](#)
- [Examples](#)
- [API reference](#)
- [SDK](#)
- [Tools](#)
- [Community](#)

Acknowledgments

Technologies

- [Polygon Network](#)
- [Supabase](#)
- [Pinata](#)
- [Paystack](#)
- [NextAuth.js](#)
- [Prisma](#)
- [Hardhat](#)
- [TypeScript](#)

Community

- [Contributors](#)
- [Users](#)
- [Testers](#)
- [Reviewers](#)
- [Supporters](#)
- [Partners](#)
- [Sponsors](#)
- [Advisors](#)

Tools

- [Development](#)
- [Testing](#)
- [Deployment](#)
- [Monitoring](#)

- Documentation
- Security
- Performance
- Support